

CI/CD and GitOps Setup on Minikube

Using Tekton and ArgoCD

A Minikube-based deployment of a Python backend, a Java backend, and an Angular frontend, built via an in-cluster Tekton CI pipeline and deployed through ArgoCD GitOps automation

Prepared by: Aaditya Vikram

Date: August 8, 2026

1. Overview

This document describes the complete design, implementation, and verification of a Continuous Integration and GitOps-based Continuous Deployment workflow on a local Minikube cluster. The assignment required deploying three applications — a Python backend, a Java backend, and an Angular frontend — using Tekton for the build pipeline and ArgoCD for GitOps-based deployment, with meaningful and easily filterable logs across both the applications and the pipeline.

The solution was implemented end-to-end within a single working session, using an in-cluster build strategy (images built directly against Minikube's internal Docker daemon rather than an external registry) to keep the workflow self-contained and fast to reproduce. All source code, Kubernetes manifests, and ArgoCD Application definitions are version-controlled in a single Git repository, which is treated as the single source of truth for both the pipeline and the deployed cluster state.

1.1 Cluster and Application Topology

- Container runtime / cluster: Minikube (Docker driver), namespaces ci-cd (Tekton pipelines) and apps (deployed workloads)
- CI engine: Tekton Pipelines (+ Tekton Dashboard), installed in the tekton-pipelines namespace
- GitOps engine: ArgoCD, installed in the argocd namespace, with automated sync and self-heal enabled
- python-backend — Flask application, container port 5000
- java-backend — Spring Boot application, container port 8080
- angular-frontend — Angular application served via nginx, container port 80
- Git repository (single source of truth): https://github.com/Quantum-Bug/minikube-ci_cd.git

1.2 Architecture Summary

Git Repository (application source + Dockerfiles + k8s manifests + ArgoCD App specs)

```
-- Tekton Pipeline (runs in-cluster, namespace: ci-cd)
    git-clone -> kaniko/buildah build -> image stored in Minikube's
    internal Docker daemon (imagePullPolicy: Never, no external registry)

-- ArgoCD Application (namespace: argocd)
    watches <app-name>/ path in the same repository (main branch)
    auto-sync + self-heal -> Deployment + Service applied to namespace: apps
```

Deployed workloads (namespace: apps):

- python-backend (Flask, containerPort 5000)
- java-backend (Spring Boot, containerPort 8080)
- angular-frontend (nginx, containerPort 80, NodePort)

Logging strategy: every workload logs to stdout, per Kubernetes convention, with consistent labels (app, tier) applied to all Deployments and Pods, allowing logs for any component to be filtered directly, for example `kubectl logs -n apps -l app=python-backend`. Tekton TaskRun logs are separated per pipeline step (git-clone, build, deploy), and ArgoCD exposes sync and health status independently per Application, so pipeline health and application health can always be diagnosed separately.

1.3 Timeline

Phase	Activity
1	Minikube cluster provisioning and namespace setup
2	Tekton Pipelines and Dashboard installation
3	ArgoCD installation and UI/CLI access configuration
4	Git repository structure, skeleton applications (Python, Java, Angular) and Dockerfiles
5	Tekton Pipeline definition (clone → build → deploy) and PipelineRuns for all three applications
6	ArgoCD Application definitions, GitOps sync, and troubleshooting of manifest resolution
7	Verification, live GitOps self-heal demonstration, and log inspection

2. Environment Setup

2.1 Minikube Cluster

The cluster was started with dedicated CPU and memory allocation, and the ingress addon was enabled for future use. Two namespaces were created up front to separate pipeline execution from deployed workloads.

```
minikube start --cpus=4 --memory=8192 --driver=docker
minikube addons enable ingress
kubectl create namespace ci-cd
kubectl create namespace apps
```

An existing Minikube profile from a prior session was detected on first start; it was removed and recreated cleanly:

```
minikube delete
minikube start --cpus=4 --memory=8192 --driver=docker
```

Result: cluster provisioned successfully on Kubernetes v1.35.1 (Docker 29.2.1 runtime), with the storage-provisioner and default-storageclass addons enabled automatically, and the ingress-nginx controller enabled explicitly.

The shell's Docker CLI was then pointed at Minikube's internal Docker daemon, so that images built inside the cluster are immediately visible to the kubelet without requiring a push to an external registry:

```
eval $(minikube docker-env)
```

2.2 Tekton Pipelines Installation

```
kubectl apply --filename https://storage.googleapis.com/tekton-releases/pipeline/latest/release.yaml
kubectl apply --filename https://storage.googleapis.com/tekton-releases/dashboard/latest/release.yaml

kubectl get pods -n tekton-pipelines --watch
```

All Tekton controller, webhook, and dashboard pods reached the Running state. The dashboard was made reachable for demonstration purposes via:

```
kubectl proxy --port=8080 &  
# http://localhost:8080/api/v1/namespaces/tekton-pipelines/services/tekton-dashboard:http/proxy/
```

2.3 ArgoCD Installation

```
kubectl create namespace argocd  
kubectl apply -n argocd -f https://raw.githubusercontent.com/argoproj/argo-cd/stable/manifests/install.yaml  
  
kubectl get pods -n argocd --watch
```

Once all ArgoCD components (application-controller, applicationset-controller, dex-server, notifications-controller, redis, repo-server, and server) reported Running, the UI was exposed locally and the initial administrator credentials were retrieved:

```
kubectl -n argocd port-forward svc/argocd-server 8081:443 &  
kubectl -n argocd get secret argocd-initial-admin-secret -o jsonpath="{.data.password}" | base64 -d; echo
```

The ArgoCD UI was accessed at <https://localhost:8081> (self-signed certificate, accepted for local development) using the admin account.

3. Git Repository and Application Structure

A single public GitHub repository (Quantum-Bug/minikube-ci_cd) was used to hold the source code for all three applications, together with their Kubernetes manifests and the ArgoCD Application definitions. Each application occupies its own top-level directory, keeping the Tekton app-path parameter and the ArgoCD source path identical and simple.

```
python-backend/  
  Dockerfile      (EXPOSE 5000)  
  app.py  
  requirements.txt  
  deployment.yaml (added in Section 5.2)  
  service.yaml    (added in Section 5.2)  
  
java-backend/  
  Dockerfile      (EXPOSE 8080)  
  Main.java  
  deployment.yaml  
  service.yaml  
  
angular-frontend/  
  Dockerfile      (EXPOSE 80)  
  angular.json, package.json, tsconfig*.json, src/, public/  
  deployment.yaml  
  service.yaml  
  
argocd-apps/  
  python-backend.yaml, java-backend.yaml, angular-frontend.yaml  
  
pipeline/  
  Tekton Pipeline and Task definitions
```

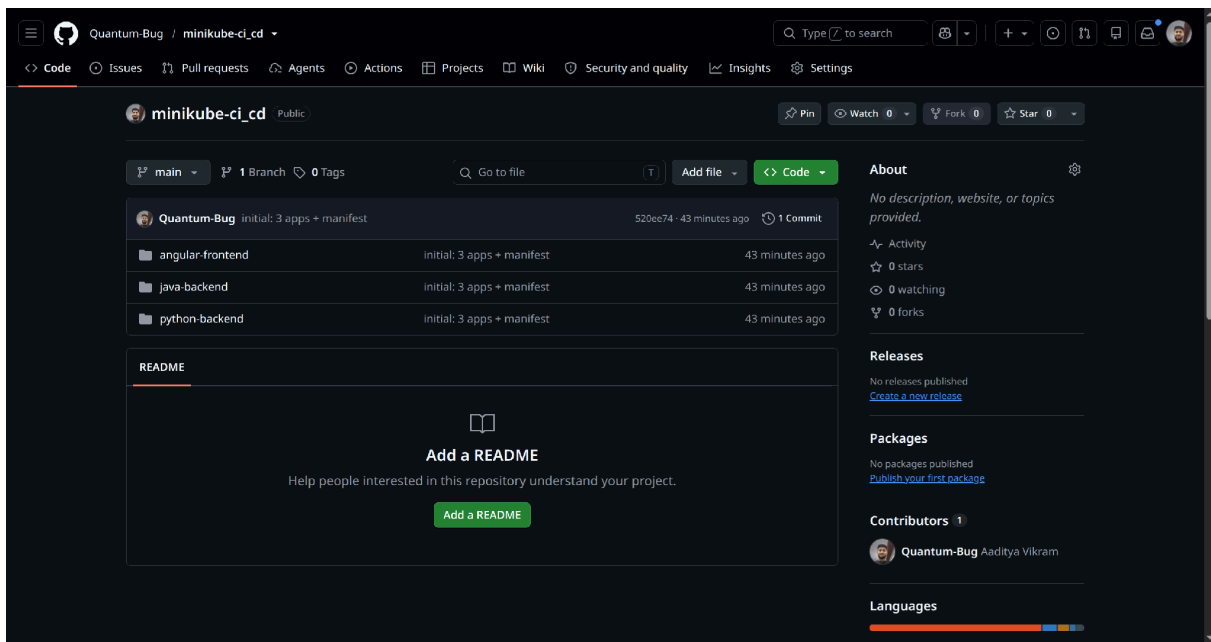


Figure 1: GitHub repository root, showing the angular-frontend, java-backend, and python-backend directories after the initial commit.

4. Tekton CI Pipeline: Build and Deploy

A single reusable Pipeline definition (build-deploy-pipeline) was created with parameters for the Git repository URL, the application's sub-path within the repository, the resulting image name, and the target Deployment name. This allowed the same Pipeline to be reused for all three applications purely by varying the PipelineRun parameters, rather than duplicating pipeline logic per application.

A dedicated ServiceAccount (tekton-deployer) was bound to the cluster with the permissions required to build container images and interact with the apps namespace.

4.1 Triggering the Pipeline for All Three Applications

PipelineRuns were generated programmatically for each application using a shell loop, referencing the shared Pipeline and passing the correct source path, image name, and deployment name per application:

```
REPO_URL="https://github.com/Quantum-Bug/minikube-ci_cd.git"

for app in "python-backend python-backend python-backend" \
  "java-backend java-backend java-backend" \
  "angular-frontend angular-frontend angular-frontend"; do
  set -- $app
  path=$1; image=$2; deploy=$3
  cat <<EOF | kubectl create -f -
apiVersion: tekton.dev/v1
kind: PipelineRun
metadata:
  generateName: build-$path-
  namespace: ci-cd
spec:
  taskRunTemplate:
```

```

    serviceAccountName: tekton-deployer
  pipelineRef:
    name: build-deploy-pipeline
  params:
    - name: repo-url
      value: "$REPO_URL"
    - name: app-path
      value: "$path"
    - name: image-name
      value: "$image"
    - name: deployment-name
      value: "$deploy"
EOF
done

```

4.2 Troubleshooting: Tekton v1 API Field Relocation

The first attempt to submit the PipelineRuns above failed under strict schema validation:

```
Error from server (BadRequest): error when creating "STDIN": PipelineRun in version "v1" cannot be handled as a PipelineRun: strict decoding error: unknown field "spec.serviceAccountName"
```

Root cause: in the Tekton v1 PipelineRun API, the executing ServiceAccount is no longer specified at `spec.serviceAccountName`; it moved under `spec.taskRunTemplate.serviceAccountName`. The manifest above already reflects the corrected field placement, which resolved the error on resubmission.

4.3 Pipeline Execution and Result

All three PipelineRuns transitioned from Running to Succeeded after their base images were pulled for the first time:

NAME	SUCCEEDED	REASON	STARTTIME	COMPLETIONTIME
build-angular-frontend-sg4jg	True	Succeeded	5m38s	86s
build-java-backend-7rjzn	True	Succeeded	5m38s	2m39s
build-python-backend-9tnmz	True	Succeeded	5m38s	3m29s

The resulting images were confirmed present in Minikube's internal Docker daemon, ready to be referenced by Kubernetes Deployments without any registry push/pull step:

```
eval $(minikube docker-env)
docker images | grep -E "python-backend|java-backend|angular-frontend"
```

5. ArgoCD GitOps Deployment

5.1 ArgoCD Application Definitions

One ArgoCD Application resource was generated per component, all pointing at the same repository and branch, each scoped to its own application directory, with automated sync, pruning, and self-heal enabled:

```
mkdir -p ~/gitops-project/argocd-apps
```

```

for app in python-backend java-backend angular-frontend; do
  cat > ~/gitops-project/argocd-apps/$app.yaml << EOF
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: $app
  namespace: argocd
spec:
  project: default
  source:
    repoURL: https://github.com/Quantum-Bug/minikube-ci_cd.git
    targetRevision: main
    path: $app
  destination:
    server: https://kubernetes.default.svc
    namespace: apps
  syncPolicy:
    automated:
      prune: true
      selfHeal: true
    syncOptions:
      - CreateNamespace=true
EOF
done

kubectl apply -f ~/gitops-project/argocd-apps/

```

5.2 Troubleshooting: Source Path Mismatch

The first sync attempt failed for every Application with a comparison error:

```

Message: Failed to load target state: failed to generate manifest for source 1 of 1:
rpc error: code = Unknown desc = Manifest generation error (cached):
manifests/python-backend: app path does not exist

```

Diagnosis: the ArgoCD Application specs referenced a path prefix of `manifests/<app-name>`, but `git ls-files` confirmed that the repository actually stores each application directly at its own top-level path (e.g. `python-backend/`, `java-backend/`, `angular-frontend/`), with no intermediate `manifests/` directory. The initial commit had placed the content at the repository root rather than under a `manifests/` subfolder.

Resolution: the `path` field in each Application spec was corrected to reference the top-level application directory directly, and the Applications were reapplied:

```

cd ~/gitops-project/argocd-apps
sed -i 's|path: manifests|path: |' *.yaml
kubectl apply -f .

```

At this point, `python-backend` and `java-backend` transitioned to `Synced / Healthy`. Deployment manifests, however, were still missing from all three application directories — this is addressed in Section 5.3.

5.3 Troubleshooting: Missing Kubernetes Manifests

Even with the correct path, `kubectl get pods -n apps` returned no resources. Inspection of each application directory confirmed that only application source code and a `Dockerfile` were present — no `deployment.yaml` or `service.yaml` existed for any of the three applications, so ArgoCD had nothing to apply.

A `Deployment` and a matching `Service` were authored for each application, using `imagePullPolicy: Never` so that the images already built by Tekton into Minikube's internal Docker daemon are used directly, with no external registry round-trip:

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: python-backend
  namespace: apps
  labels: { app: python-backend, tier: backend }
spec:
  replicas: 1
  selector: { matchLabels: { app: python-backend } }
  template:
    metadata: { labels: { app: python-backend, tier: backend } }
    spec:
      containers:
        - name: python-backend
          image: python-backend:latest
          imagePullPolicy: Never
          ports: [{ containerPort: 5000 }]
---
apiVersion: v1
kind: Service
metadata:
  name: python-backend
  namespace: apps
  labels: { app: python-backend }
spec:
  selector: { app: python-backend }
  ports: [{ port: 5000, targetPort: 5000 }]
  type: ClusterIP
```

Equivalent manifests were created for `java-backend` (`containerPort 8080`, `ClusterIP`) and `angular-frontend` (`containerPort 80`, `NodePort`, to allow direct browser access via minikube service). All manifests were committed and pushed:

```
git add .
git commit -m "Add k8s deployment and service manifests for all apps"
git push origin main
```

5.4 Troubleshooting: JSON Parsing Error on the Angular Application

After the manifests were added, `python-backend` and `java-backend` deployed successfully, but `angular-frontend` remained stuck in an `Unknown` sync state. The ArgoCD repo-server logs isolated the exact cause:

```
grpc.error: "Manifest generation error (cached): rpc error: code = FailedPrecondition
desc = Failed to unmarshal \"tsconfig.app.json\": invalid character '/' looking for
beginning of value"
```


Diagnosis: by default, ArgoCD's Directory source type attempts to parse every YAML/JSON file in the application path as a Kubernetes manifest — not only deployment.yaml and service.yaml. The Angular project directory also contains tsconfig.app.json, angular.json, and package.json, and TypeScript's tsconfig files permit // comments, which are not valid strict JSON. ArgoCD's parser therefore failed while trying to treat a build-tool configuration file as a Kubernetes resource.

Resolution: the ArgoCD Application source was restricted to an explicit include glob so that only the intended manifest files are considered, applied to all three Applications as a defensive measure:

```
kubectl patch application angular-frontend -n argocd --type merge -p
'{"spec":{"source":{"directory":{"include":"{deployment.yaml,service.yaml}"}}}}'

kubectl patch application python-backend -n argocd --type merge -p
'{"spec":{"source":{"directory":{"include":"{deployment.yaml,service.yaml}"}}}}'

kubectl patch application java-backend -n argocd --type merge -p
'{"spec":{"source":{"directory":{"include":"{deployment.yaml,service.yaml}"}}}}'
```

5.5 Troubleshooting: Repo-Server Manifest Generation Timeout

A separate, transient issue was observed after a subsequent hard refresh: an application controller event reported a manifest-generation timeout rather than a parsing error:

```
Warning OperationCompleted Sync operation to <revision> failed: ComparisonError:
Failed to load target state: failed to generate manifest for source 1 of 1:
rpc error: code = DeadlineExceeded desc = context deadline exceeded
```

This was investigated systematically before being resolved: node resource allocation (kubectl describe node), host memory and CPU (free -h, nproc), and outbound connectivity from the repo-server pod to GitHub were each checked in turn. Host resources were confirmed ample (16 cores, 8.5 GiB free memory) and network connectivity from a disposable test pod returned HTTP 200 from github.com, ruling out both resource starvation and connectivity as root causes; the applicationset-controller pod's repeated restarts pointed instead to transient pod-level instability rather than host capacity. A targeted restart of the repo-server component cleared the stuck internal state:

```
kubectl rollout restart deployment argocd-repo-server -n argocd
kubectl rollout status deployment argocd-repo-server -n argocd
```

Following the restart and a subsequent manual sync, all three Applications reached Synced / Healthy.

5.6 Final Deployment State

NAME	SYNC STATUS	HEALTH STATUS
angular-frontend	Synced	Healthy
java-backend	Synced	Healthy
python-backend	Synced	Healthy

NAME	READY	STATUS	RESTARTS	AGE
angular-frontend-689c9d966c-c27jd	1/1	Running	0	31s
java-backend-768bb6d7c7-6nj4t	1/1	Running	0	21s
python-backend-97bccd578-2dk5j	1/1	Running	0	5m8s

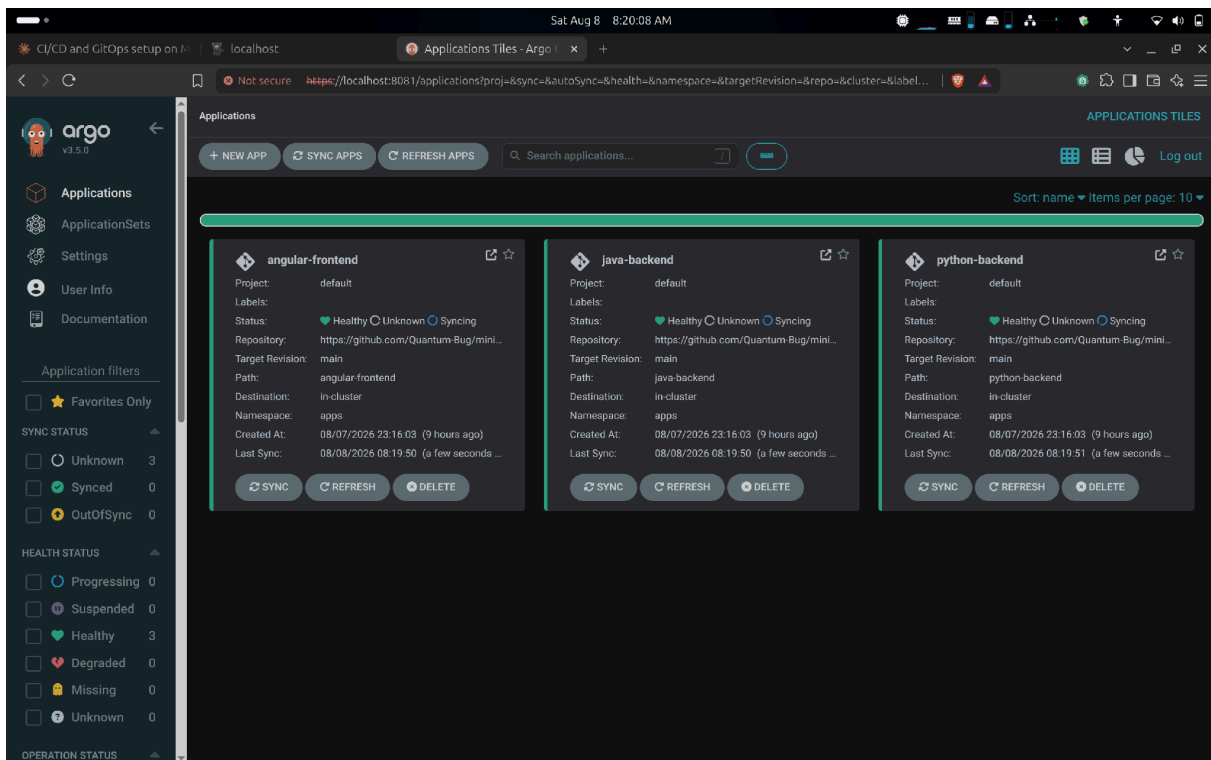


Figure 2: ArgoCD Applications Tiles view, showing angular-frontend, java-backend, and python-backend, each with repository, target revision, source path, destination namespace, and last-sync time.

6. GitOps Self-Healing: Live Verification

To demonstrate that Git is genuinely the single source of truth for cluster state — rather than the cluster being managed by any direct kubectl command — the replica count of python-backend was changed only in the manifest committed to Git, with no kubectl apply or kubectl scale issued against the cluster at any point.

6.1 Procedure

- Edit python-backend/deployment.yaml locally, changing spec.replicas from 1 to 2.
- Commit and push the change to the main branch:

```
git add python-backend/deployment.yaml
git commit -m "Scale python-backend to 2 replicas"
git push origin main
```

- Trigger a hard refresh so ArgoCD re-compares desired state against the new commit immediately, rather than waiting for its default polling interval:

```
kubectl patch application python-backend -n argocd --type merge -p
'{"metadata":{"annotations":{"argocd.argoproj.io/refresh":"hard"}}}'
```

- Observe the Pods in the apps namespace:

```
kubectl get pods -n apps -l app=python-backend --watch
```

6.2 Result

NAME	READY	STATUS	RESTARTS	AGE
python-backend-97bdd578-2dk5j	1/1	Running	0	10m
python-backend-97bdd578-jj4wv	1/1	Running	0	37s

A second python-backend Pod was created automatically, 37 seconds after the ArgoCD refresh, purely as a consequence of the Git commit. This confirms the GitOps control loop end-to-end: the desired state lives in Git, ArgoCD continuously reconciles the live cluster state against it, and any drift-free change is applied automatically under the selfHeal policy configured in Section 5.1 — with no manual cluster mutation performed.

7. Post-Deployment Verification

7.1 Services

```
kubectl get svc -n apps
```

7.2 Application Reachability

```
kubectl port-forward -n apps svc/python-backend 5000:5000 &
curl localhost:5000
```

```
kubectl port-forward -n apps svc/java-backend 8080:8080 &
curl localhost:8080
```

```
minikube service angular-frontend -n apps --url
```

7.3 Log Inspection (by label)

As specified in Section 1.2, consistent app labels allow logs for any component to be retrieved independently of pod naming/restarts:

```
kubectl logs -n apps -l app=python-backend --tail=20
kubectl logs -n apps -l app=java-backend --tail=20
kubectl logs -n apps -l app=angular-frontend --tail=20
```

Tekton pipeline-step logs remain independently inspectable per PipelineRun and per step (git-clone, build, deploy), and ArgoCD's sync/health status is visible per Application in both the CLI and the UI, satisfying the requirement for clearly separable, meaningful logging across the pipeline and the applications.

8. Project Structure and Navigation Guide

The entire project is organized under the Git repository root, with mirrored local working copies under ~/gitops-project/ and ~/mongo-cluster/-style separation between application code and operational tooling.

```
minikube-ci_cd/          # Git repository root (source of truth)
|-- python-backend/
|   |-- Dockerfile, app.py, requirements.txt
|   |-- deployment.yaml, service.yaml
|-- java-backend/
```

```
| |-- Dockerfile, Main.java
| |-- deployment.yaml, service.yaml
|-- angular-frontend/
| |-- Dockerfile, angular.json, package.json, src/, public/
| |-- deployment.yaml, service.yaml
|-- argocd-apps/
| |-- python-backend.yaml, java-backend.yaml, angular-frontend.yaml
`-- pipeline/
    |-- Tekton Pipeline and Task definitions, ServiceAccount (tekton-deployer)
```

8.1 Everyday Operations

Starting and stopping the cluster

```
minikube start
eval $(minikube docker-env)
minikube stop
```

Checking pipeline and application status

```
kubectl get pipelineruns -n ci-cd
kubectl get pods -n ci-cd
kubectl get applications -n argocd
kubectl get pods -n apps
```

Triggering a new build

Resubmit a PipelineRun for the relevant application using the loop pattern in Section 4.1, or via the Tekton Dashboard.

Forcing an immediate GitOps sync

```
kubectl patch application <app-name> -n argocd --type merge -p
'{"metadata":{"annotations":{"argocd.argoproj.io/refresh":"hard"}}}'
```

Accessing the dashboards

- Tekton Dashboard: via kubectl proxy, at the tekton-dashboard service proxy path (Section 2.2)
- ArgoCD UI: <https://localhost:8081> (via kubectl -n argocd port-forward svc/argocd-server 8081:443)

8.2 Access Summary

Component	Namespace	Access
Tekton Pipelines	tekton-pipelines / ci-cd	kubectl proxy (Dashboard)
ArgoCD	argocd	https://localhost:8081 , user admin
python-backend	apps	ClusterIP :5000 (port-forward)
java-backend	apps	ClusterIP :8080 (port-forward)
angular-frontend	apps	NodePort (minikube service --url)

Security note: this environment is configured for local demonstration and development purposes only. A production-equivalent deployment would additionally require TLS termination, RBAC hardening beyond the default ArgoCD/Tekton service accounts, an external (rather than in-cluster) container registry, and secret management for any credentials used by the pipeline or applications.

9. Lessons and Recommendations

- Tekton v1's PipelineRun schema moved serviceAccountName under spec.taskRunTemplate; strict decoding will reject the pre-v1 field placement outright rather than warning.
- An ArgoCD Application's source.path must match the repository's actual directory layout exactly; git ls-files is the fastest way to confirm real paths when a ComparisonError reports “app path does not exist.”
- ArgoCD's default Directory source parses every YAML/JSON file under the source path as a candidate Kubernetes manifest. Front-end/build-tool config files (tsconfig.*.json, angular.json, package.json) should be excluded explicitly via source.directory.include, both to fix parsing errors and as a general defensive practice.
- A ComparisonError of DeadlineExceeded on the repo-server is not necessarily a resource-capacity issue; ruling out node allocation, host memory/CPU, and outbound network connectivity first (in that order) narrows the diagnosis quickly, and a targeted argocd-repo-server rollout restart resolves transient stuck states without needing to touch the rest of the ArgoCD installation.
- Building images against Minikube's internal Docker daemon (imagePullPolicy: Never) removes the need for an external registry entirely for local development and demonstration, at the cost of not reflecting a production image-distribution flow.
- The most convincing GitOps verification is a change that touches only Git — no kubectl apply, scale, or edit — followed by observing the cluster reconcile itself; this was demonstrated directly in Section 6.

Appendix A: Reference Commands

A.1 Full Cluster Bring-Up (from a clean checkout)

```
minikube start --cpus=4 --memory=8192 --driver=docker
minikube addons enable ingress
kubectl create namespace ci-cd
kubectl create namespace apps
eval $(minikube docker-env)

kubectl apply --filename https://storage.googleapis.com/tekton-releases/pipeline/latest/release.yaml
kubectl apply --filename https://storage.googleapis.com/tekton-releases/dashboard/latest/release.yaml

kubectl create namespace argocd
kubectl apply -n argocd -f https://raw.githubusercontent.com/argoproj/argo-cd/stable/manifests/install.yaml

kubectl apply -f argocd-apps/
```

A.2 Health Check, One-Liner

```
kubectl get pods -n tekton-pipelines && \
kubectl get pods -n argocd && \
kubectl get pipelineruns -n ci-cd && \
kubectl get applications -n argocd && \
```

```
kubectl get pods -n apps
```